Java Programming 1
Lecture 10
Simple Decisions

Midterm Exam Review
Boolean Values, Relational Operators
and the if Statement

Express Yourself

- Use OpenOffice Writer to create a new document
- Save the file as LastFirst_ic10
  - Replace LastFirst with your actual name
- Put your name and today's date at the top of the sheet
- Title it "CS 170 In-class Exercise 10"
- **Exercise 1**: Open BlueJ and create a new project named ic10. Copy the folder named cards from Q: and drop it in your ic12 folder. Create a class named EX10A using the ACM Graphics template. Shoot me a screen-shot.

George Boole

- 1815-1899 : Developed the mathematical connection between Algebra and logic that underlies computing

The boolean Primitive Type

- The last of the four primitive types is **boolean**
  - (Note that there is also a **Boolean** wrapper class)
  - The **boolean** type has two possible values
  - **boolean** **literals** are written as **true** and **false**
    - Remember, Java is case sensitive
- Such literals can be used as method arguments

```
card1.setVisible(false);
myRect.setFilled(true);
```

Creating Some Cards

- We can use the **GImage** class to represent cards
  - We'll make both of these instance variables
  - Use the "Ace of Spades" to initialize the first card
  - Use the "Back" image for the second card

```
private GImage card1 = new GImage("cards/as.gif");
private GImage card2 = new GImage("cards/b.gif");
```

- **Exercise 2**: Shoot a screen-shot of your code

Position and Display

- In the **run()** method, position and display the cards
  - Move **card1** 15 to the right and 15 down
  - Set the location of **card2** to the corner of **card1**
  - Move it right by 25 and down by 10
  - Add both of them to the surface of the app

```
card1.move(15, 15);
card2.setLocation(card1.getLocation());
card2.move(25, 10);
add(card1);
add(card2);
```

Exercise 3: Compile, run and snap a picture.

Responding to Mouse Events

- A **GraphicsProgram** can "listen" for mouse events
  - Then, you write a method to respond to the event
  - Add these two lines to your program

```
import java.awt.event.*;  // at top of file
addMouseListeners(); // at end of run()
```

- There are several **kinds** of mouse events
  - Clicked, Moved, Entered, Exited, Dragged, etc..

Express Yourself

- Let's add a method to handle a mouse click

```
public void mouseClicked(MouseEvent e)
{
}
```

- **Exercise 4**: All ACM graphical objects have a **setVisible(boolean)** method. Passing **true** makes the object visible, while passing **false** makes it invisible. In your **mouseClicked()** method, make **card2** invisible. Run your program, click and shoot a screen-shot.

boolean Variables and Methods

- You create **boolean** variables like other variables

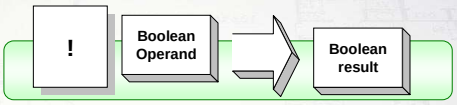
```
boolean visible = false;
```

- Methods can also produce **boolean** values
  - Normally store result in a **boolean** variable

```
visible = aLabel.isVisible();
enabled = aButton.isEnabled();
showing = aWindow.isShowing();
```

The boolean NOT Operator

- The unary NOT symbol is the exclamation point !



  - It appears in front of its boolean operand
  - It produces the reverse of the expression it operates on

```
boolean visible = true;
boolean what = !visible;     // false
visible = !visible;          // toggle
```

Express Yourself

- Exercise 5: All ACM graphical objects have an isVisible() accessor method that returns true if the object is visible and false otherwise. In your mouseClicked() method, call isVisible on card2, and save the result in a boolean variable. Then use the NOT operator to pass the opposite value to card2.setVisible(). Run your program, make sure you can "toggle" the card, and shoot a screen-shot.

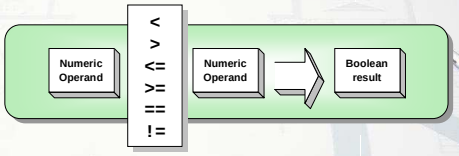
The Relational Operators

- Most boolean values are produced by comparing two values using the relational operators
- Six operators: <, <=, >, >=, ==, !=
  - Relationship between two values? true or false

Example	Result
a < b	true if a is less than b; otherwise false
a <= b	true if a is less than or equal to b; otherwise false
a > b	true if a is greater than b; otherwise false
a >= b	true if a is greater than or equal to b; otherwise false
a == b	true if a is equal to b, otherwise false
a != b	true if a is not equal to b, otherwise false

Primitive Relations

- You can compare most primitive **numeric** types, including **char**, but not **boolean**, without problem
  - You don't have to worry about precision or size



Express Yourself

- **Exercise 6**: Here are some primitive variables along with some relational expressions. Type these variables into CodePad and then try out each of the expressions below. Shoot a screen-shot of the result

```
b > sh
c < a
f == a
f < a
c > a
```

```
// Variables
byte b = 2;
short sh = 3300;
double a = 2.34;
float f = 2.34F;
char c = 'A';
```

Variables of several different primitive types

Floating-point Relations

- Generally, you **shouldn't use** == or != with reals
- Problems with **mixed type** comparisons
  - With mixed type, **2.34** and **2.34F** are NOT equal
  - They have different bit patterns

```
public static void main(String[] args)
{
    // 1. Comparing different types
    float a = 2.34F;
    double b = 2.34;

    System.out.println("a == b is " + ( a == b));
    System.out.println("a > b is " + (a > b));
    System.out.println("a < b is " + (a < b));
```

Floating Point Calculations

- Even within the same type logically equivalent binary calculations may cause the results to differ
  - Multiply **.1 * 10.0**, and then add **.1** ten times
  - Mathematically equivalent, but differ in Java

```
// 2. Comparing two mathematically equivalent values
double c = .1 * 10.0;
double d = .1 + .1 + .1 + .1 + .1 + .1+ .1+ .1+ .1+ .1;

// 3. Another example (from Cay Horstmann's Big Java, p. 196)
double root = Math.sqrt(2);          // square root of 2
double result = root * root - 2;     // subtract 2 from 2
System.out.println("Subtracting 2 from 2 gives " + result);
```

Safe Real Comparisons

- There is no "one-size-fits-all" solution
  - For "dollars and cents" (accurate to a penny) do this

```
(int)(( a + .005) * 100) == (int)(( b + .005) * 100)
```

- A more "general purpose" algorithm looks like this:

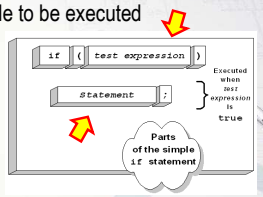
```
// 4. A check to see if numbers are "close enough"
final double EPSILON = 10E-14;
boolean areEqual = Math.abs(c - d) <= EPSILON;
System.out.println("c and d 'areEqual' ? " + areEqual);
System.out.println("result is effectively 0 ? " +
                   (result <= EPSILON));
```

Introducir

- One type of **Flow of Control** statement in computer programs
- Selection acts as "highway divider" in your code
  - Causes conditional execution of alternative code
  - Also known as "branching"
  - Uses **boolean** expressions from **relational** operators
  - If your salary is less than $ 25,000 then
    Calculate tax based on 15% marginal rate
    otherwise
    Calculate tax based on 15% base
    and a 28% marginal rate

The if Statement

- Like classes and methods, the **if** has two parts
  - The header contains a **boolean** condition
  - The body contains code to be executed
- If the condition is **true**, then the code in the body is executed
- If the condition is **false**, the code in the body is skipped

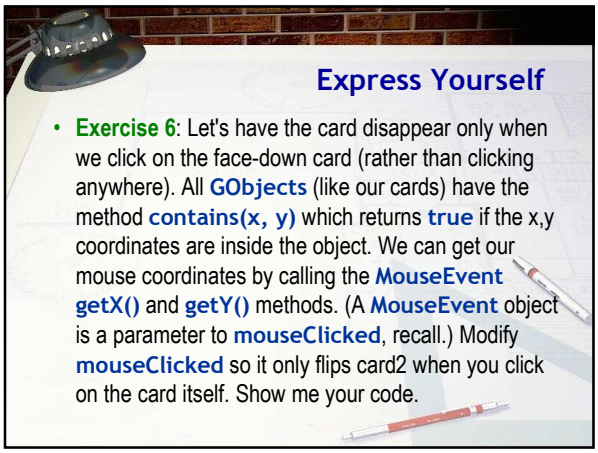


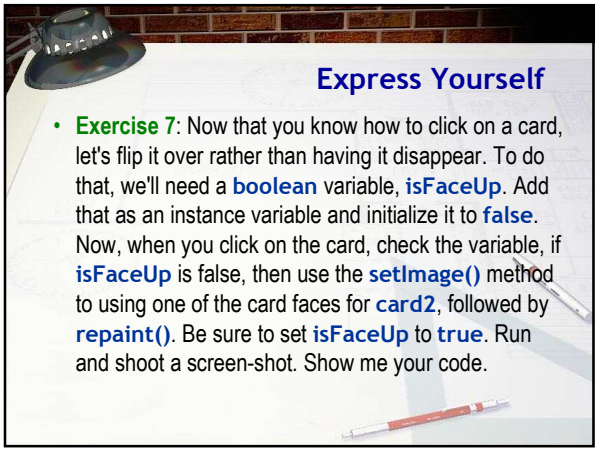
Syntax Points

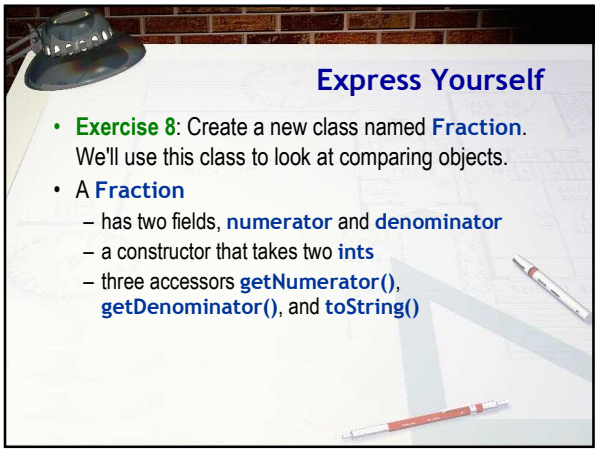
- The keyword **if** must be in lower-case
  - If or IF won't work
- Place a **boolean** expression in the parentheses
  - The parentheses are required (unlike VB or Pascal)
  - You can't use a numeric value like C
- **Don't** put a semicolon after the condition
  - Common error, **not** be caught by the compiler
  - Real body always executed when this happens
- The body consists of a **single** statement

A Few Fine Points About if

- Many languages use the **=** symbol for equality
  - **if (x = 10)** won't compile in Java
  - **if (ok = true)** does compile, but is **always** incorrect
- **Never** compare against the literals **true** or **false**
  - **if (ok == true)** is very very poor style
- Many languages use **<>** to mean not-equal
  - **if (x <> 10)** won't compile in Java
- Use **>=** rather than **=>** which doesn't compile

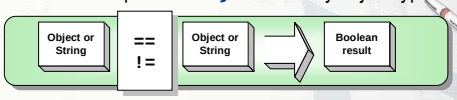

Express Yourself
• Exercise 6: Let's have the card disappear only when we click on the face-down card (rather than clicking anywhere). All GObjects (like our cards) have the method contains(x, y) which returns true if the x,y coordinates are inside the object. We can get our mouse coordinates by calling the MouseEvent getX() and getY() methods. (A MouseEvent object is a parameter to mouseClicked, recall.) Modify mouseClicked so it only flips card2 when you click on the card itself. Show me your code.


Express Yourself
• Exercise 7: Now that you know how to click on a card, let's flip it over rather than having it disappear. To do that, we'll need a boolean variable, isFaceUp. Add that as an instance variable and initialize it to false. Now, when you click on the card, check the variable, if isFaceUp is false, then use the setImage() method to using one of the card faces for card2, followed by repaint(). Be sure to set isFaceUp to true. Run and shoot a screen-shot. Show me your code.


Express Yourself
• Exercise 8: Create a new class named Fraction. We'll use this class to look at comparing objects.
• A Fraction
– has two fields, numerator and denominator
– a constructor that takes two ints
– three accessors getNumerator(), getDenominator(), and toString()

Object Relations

- When comparing objects and **Strings** you can only use the **==** and **!=** operators, not **>**, **<**, **>=**, or **<=**
  - You should not mix different object types
  - Don't compare a **Button** with a **Label**, for instance
  - You can compare an **Object** with any object type



Express Yourself

- **Exercise 9**: Here are some CodePad exercises
  - Create three **Fraction** objects (**a**, **b**, **c**) : 1/2, 1/2, 1/3
  - Create two **Strings** (s1, s2), containing "Hi Mom"
  - What happens with the following comparisons:
    - **s1 == s2**
    - **s1 > s2**
    - **a == c**
    - **a > c**
    - **a == b**
    - **a != s1**

Shoot me a screen-shot.

The instanceof Operator

- A 7th relational operator works **only** with objects
  - Uses keyword **instanceof** instead of a symbol
  - **true** if **object** on left is instance of **class** on right
  - Also if object belongs to a subclass of class on right

```
Button b = new Button("Hi");
Menu m = new Menu("File");
boolean ans;

ans = b instanceof Bu
ans = b instanceof Co
ans = m instanceof Me
ans = m instanceof Co
```

Exercise 10: See if s1 is an instance of Fraction, String, Object. See if a is an instance of Number, Object.

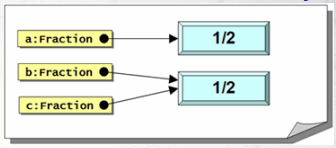
Simple Equality

- With objects, == measures **identity** or **simple equality**
  - Two variables refer to the **same** object or **String**
  - Here's an example with three **Fraction** variables

```
Fraction a = new Fraction(1, 2);
Fraction b = new Fraction(1, 2);
Fraction c = b;
```

Fractions in Memory

- In memory, the **Fractions** and variables look like this



- Here's what happens when we compare these:

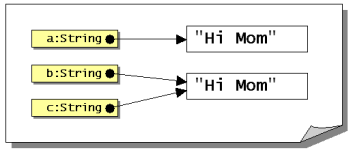
```
a == b; // false - different Fraction objects
b == c; // true - same Fraction
a == c; // false - different Fractions
```

Strings Comparisons

- **Strings** are objects; comparisons work the same way
- If you look at this code:

```
String a = "Hi Mom";
String b = "Hi Mom";
String c = b;
```

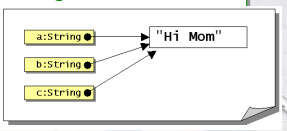
- You'd expect to see this in memory:



Strings in Memory

- With the same boolean expressions, though:

```
a == b; // true??? - Why?
b == c; // true - same String
a == c; // true???
```



- The reason the **Strings** are identical is because **Strings** are **immutable**, so memory really looks like this:

Value Equality

- Value equality: two objects have the **same contents**
  - This varies on a type-by-type basis
- Types that are comparable in this way will overload the **Object** method called **equals()**
- **Exercise 11**: Try this in CodePad:

```
String name = "Freddie".substring(0, 4);
name == "Fred";
name.equals("Fred");
```

Comparing Strings

- Methods for **comparing Strings**
  - **s == s2**
    - Compares the addresses in the variables s and s2
    - Returns true if s and s2 refer to the same string
  - **s.equals(s2), s.equalsIgnoreCase(s2)**
    - Returns true if two strings have the same contents
    - Lexical, not dictionary comparison
  - **s.compareTo(s2)**
    - Compares contents of **String** s and the **String** s2. Returns 0 if equal, positive if s is "greater" than S2, negative if less

Express Yourself

- **Exercise 12**: Add an **equals()** method to the **Fraction** class. This is a predicate method (that is, it will return **true** or **false**). It takes one **Fraction** as a parameter. It should return **true** if two objects have the same address, and it should also return **true** if two **Fraction**s have both the same numerator and same denominator.
- **Exercise 13**: Add a test program. Create three **Fractions** (1/2, 1/2, 1/3). Test your **equals()** method several different ways. Shoot a screen-shot.

What Card Was Picked?

- **Import** file **Ex10B.java** from Q:, then open it.
  - A **GRect** table with two **GImage** cards face down
  - Want to flip over the card that was clicked
  - Use **getElementAt(x, y)** method to get a reference to the object that was clicked.
    - Save in a **Object** variable named **picked**

```
int x = e.getX();
int y = e.getY();
Object picked = getElementAt(x, y);
```

Simple If Actions

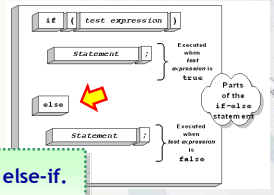
- You can compare an **Object** against **any** object type
  - Compare **picked** to **card1** using an **if** statement
    - If **true**, use **setImage()** to "flip" the card over

```
if (picked == card1)
   card1.setImage(getRandomCard());
```

  - Repeat for the **card2** variable
- **Exercise 14**: Run and make sure you can flip each card by clicking on it. Copy and paste your code.

A Small Problem

- Your program is **misleading** and **not very efficient**
  - If you pick **card1**, your code still checks **card2**
  - Not necessary because choice is **mutually exclusive**
- Java's **if** has an **else** portion that can be efficiently used for such **either/or** conditions



Exercise 15: Modify to use else-if. Snap a picture of your code

Blocks and Indentation

- Most of the time, you need to carry out **several** actions inside the body of an **if** statement, not just one
- To do this, surround the body with braces (a "block")

```
if ( amountSold <= 35000)
{
    bonusPct = .035;
    bonusAmt = amountSold * bonusPct;
}
```

- Indent the code inside the block by one tab stop

Why Use Braces?

- Braces are not ***required*** for a single-line **if** statement
- Still a good idea, even when not required
  - Helps avoid mistakes when modifying code

```
if (amountSold <= 35000)
    bonusPct = .035;
else
    bonusPct = .075;
    bonusAmount += 100;
bonusAmount += amtSold * BonusPct;
```

The Phantom Semicolon

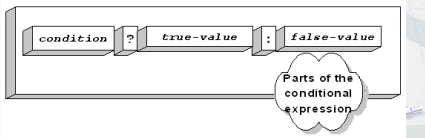
- Common mistake: a semicolon after the condition

```
if (filesAreAllBackedUp);
{
    formatTheHardDrive();
}
```

- Because of the semicolon, this fragment of code says:
  - 1. Check to see if files are backed up
  - 2. If they are, don't do anything (the **null** statement)
  - 3. Format the hard-drive regardless of the backup

The Selection Operator

- You can't use **if-else** in a formula
  - An **if** is a **statement**, not an expression
  - Means you often need an additional variable
- Selection operator is like **if-else as an expression**
  - AKA the **ternary**, **tertiary** or **conditional** operator
  - Works similar to the **=IF()** function in Excel



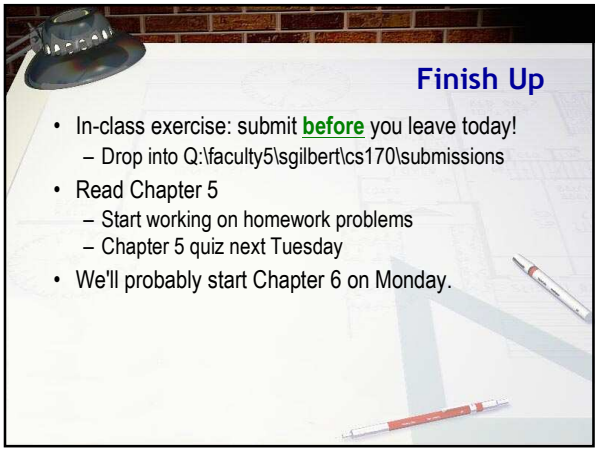
Conditional Operator Syntax

- The first operand must be a boolean condition
  - Followed by a **?**
- The true and false parts are separated by a colon
  - Both parts must be **compatible** types

```
int value = (amt > 0) ? 10 : 30;    // OK integers
int value = (amt > 0) ? 10 : "Hi";  // No, different types
```

- Can "nest", but quickly becomes unreadable


The Cigar Party
• Log into your JavaBat account
– In the Logic problems, locate Cigar Party
• Exercise 16: Spend 5 minutes using if and else to try and solve the Cigar Party problem shown here.
– When squirrels get together for a party, they like to have cigars. A squirrel party is successful when the number of cigars is between 40 and 60, inclusive. Unless it is the weekend, in which case there is no upper bound on the number of cigars. Return true if the party with the given values is successful, or false otherwise.
• Snap a picture of your best run


Finish Up
• In-class exercise: submit before you leave today!
– Drop into Q:\faculty5\sgilbert\cs170\submissions
• Read Chapter 5
– Start working on homework problems
– Chapter 5 quiz next Tuesday
• We'll probably start Chapter 6 on Monday.